SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **MUJAHITH M / 192312399** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MUJAHITH M

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. **Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty**

ii. **Explain how A* would handle the same situation, including how it balances cost and heuristic**

iii. **Analyze the impact of heuristic accuracy on both algorithms**

iv. **Discuss how dynamic changes (blocked paths) affect search performance**

v. **Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety**

Answer:

i. Greedy Best-First Search (GBFS). GBFS always expands the node with the smallest heuristic value, f(n) = h(n), where h(n) is the straight-line distance from the drone to the medical drop point. In the flood scenario the drone would head almost directly towards the destination, choosing whichever neighbouring waypoint looks closest.

Strengths: very fast decisions, minimum computation and memory, and it needs only local information, so it suits an urgent mission with partial real-time updates.

Weaknesses: it ignores the real travel cost g(n) (terrain, wind, rain), so it cannot prefer a safer but slightly longer route. It is neither optimal nor reliably complete. The straight-line heuristic is misleading when the direct line crosses a flooded valley or a blocked road, so the drone can be drawn into dead ends and then waste time backtracking. Under uncertainty it commits to risky paths that merely look short.

ii. A* Search. A* uses f(n) = g(n) + h(n). Here g(n) is the actual accumulated cost from the base to the current waypoint, with weather and terrain penalties added to the edge costs, and h(n) is the straight-line distance still remaining. g(n) stops the drone choosing very expensive or dangerous segments, while h(n) keeps the search pointed at the goal, so it avoids exploring in every direction. Blocked paths are treated as edges of infinite cost and removed, and the drone re-plans from its current position. Because straight-line distance never overestimates the true cost (it is admissible), A* is complete and gives the minimum-cost route. Its drawbacks are higher memory use and more computation than GBFS.

iii. Impact of heuristic accuracy. In GBFS the heuristic is the only guide, so the result depends entirely on its quality: an accurate h gives a near-direct path, while a poor h (large underestimates around floods) causes long detours and no correction is possible. In A* an accurate and admissible h reduces the number of nodes expanded and still guarantees optimality. A weak h (for example h = 0, which becomes Dijkstra) is still correct but slow because many more nodes are expanded. An overestimating (inadmissible) h makes A* faster but can lose optimality. In a flood, straight-line distance is safe but under-informative, so A* stays correct but expands more nodes, whereas GBFS may become wrong.

iv. Effect of dynamic changes. When a path is blocked the stored plan becomes invalid and the search must be repeated from the drone's current position. GBFS can re-plan very cheaply, but it may repeatedly walk into the same kind of trap. A* gives better routes after each change but costs more time per re-plan, and frequent blocking increases total delay. Changing costs also make old g-values stale. Incremental variants such as D* Lite or LPA* reuse earlier search effort and only repair the affected part of the route, so they keep re-planning fast.

v. Recommendation. A* (preferably in a replanning/weighted form such as D* Lite or Weighted A*) is the most suitable. For optimality, it uses true costs, so it finds the least-delay route. For safety, hazard and weather risk can be added to g(n), so dangerous areas are avoided. For real-time constraints, a weight $w > 1$ ($f = g + w \cdot h$) or a bounded, anytime version returns a good route quickly, and incremental re-planning copes with blocked paths. GBFS is kept only as an emergency fallback when almost no computation time remains. Thus A* gives the best balance of speed, cost and safety for medicine delivery.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. **Explain how Hill Climbing would work in this scenario and identify its limitations**

ii. **Discuss issues like local maxima, plateaus, and ridges with real-world interpretation**

iii. **Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations**

iv. **Recommend the best approach and justify your choice based on scalability and adaptability**

Answer:

Formulation as an optimization problem. State: a complete signal-timing configuration S = (cycle length, green split and offset for each of the n intersections). Objective function: minimize F(S) = w1·(total waiting time) + w2·(fuel consumption) + w3·(congestion/queue length), usually computed with a traffic simulator. Constraints: minimum and maximum green time, pedestrian crossing time, valid phase order, and priority for emergency vehicles. Neighbour move: change the timing of one intersection by a small step. Why traditional search is inefficient: the space grows exponentially (k options per signal gives k^n configurations for n signals); there is no single goal state to test, only a better or worse configuration; the path used to reach a configuration does not matter; every evaluation needs a costly simulation; and as traffic changes, the best configuration keeps moving. BFS, DFS and A* would run out of time and memory, so local/optimization search is suitable.

i. Hill Climbing. Start from the current timing plan, evaluate neighbouring plans, move to the neighbour with the lowest F(S), and repeat until no neighbour is better. It stores only the current state, is very fast, and can run continuously to adjust signals as flow changes. **Limitations:** it can stop at a local optimum, wander on plateaus, cannot follow ridges, never moves "downhill" to escape, depends on the starting plan, and gives no guarantee of the best solution.

ii. Local maxima, plateaus and ridges (real-world view). *Local optimum:* a "green wave" coordinated along one main road is good, yet changing any single signal makes it worse; a better plan needs a city-wide re-coordination, but hill climbing is stuck. *Plateau:* during steady off-peak traffic, changing a green time by a few seconds does not change waiting time, so the algorithm cannot see which direction is better and wastes time or stops. *Ridge:* adjacent signals must be changed together (offset and cycle length at the same time); each single change alone looks worse, but the combination improves traffic, so single-step moves cannot follow the narrow path.

iii. Simulated Annealing (SA) and Genetic Algorithms (GA). SA sometimes accepts a worse plan with probability e^(-ΔF/T). At high temperature T it explores widely (for example, accepting more waiting at one junction to enable corridor coordination), and as T is lowered it settles into a good solution, so it escapes local optima, plateaus and ridges. GA keeps a population of timing plans (chromosome = vector of green times and offsets), uses fitness = 1/F(S), and applies selection, crossover (combining the best timings of different corridors) and mutation (random small changes). Because many plans are explored at once, GA maintains diversity, avoids premature convergence and can evaluate plans in parallel.

iv. Recommended approach. A Genetic Algorithm, combined with a hill-climbing refinement step (a memetic approach), is best. For scalability, the population can be divided by city zone and evaluated in parallel across hundreds of intersections. For adaptability, the previous population is reused as the starting point when traffic changes, so the system re-adapts quickly instead of starting again, and the population keeps several alternative good plans. SA is simpler and uses less memory, but it is a single-path search that is sensitive to the cooling schedule and is slower to adapt. Hence GA for global optimization with hill climbing for real-time fine tuning is the best choice.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. **Explain why this problem requires an online search agent instead of offline search**

ii. **Analyze the challenges of partial observability and dynamic environments**

iii. **Describe how the rover builds and updates its internal model**

iv. **Compare online search with uninformed and informed search strategies**

v. **Justify the most suitable approach considering uncertainty, adaptability, and safety**

Answer:

i. Why online search is required. In offline search the agent computes a complete plan before acting, which needs a complete and correct model of the environment. The Mars terrain is unknown, so the map does not exist beforehand. The rover discovers states only by moving and sensing, and communication delay to Earth (several minutes each way) prevents remote control. Actions can also be irreversible (falling into a crater, getting stuck), so planning and acting must be interleaved: sense, update, decide, move, and repeat. This makes it an exploration problem that only an online search agent can solve.

ii. Challenges of partial observability and dynamic environments. *Partial observability:* sensors have limited range and noise (dust, shadows, glare), so hazards behind rocks or slopes are not seen, and different real situations look identical. The rover must keep a belief about its state and position, and localization drifts because of wheel slip and no GPS. *Dynamic environment:* dust storms, shifting sand, falling rocks, changing light, temperature and battery state mean that earlier information becomes stale. Further difficulties are limited on-board computation, dead-end and irreversible states, and the need to explore safely while still reaching the sample sites.

iii. Building and updating the internal model. The rover starts with an almost empty model. After every action it receives a percept and updates its map (an occupancy grid or graph of visited states). It records the result table result[s, a] = s', the action cost, the untried actions in each state, and the probability that a cell is a hazard (updated by Bayesian filtering). SLAM (simultaneous localization and mapping) keeps the position estimate consistent. Predicted outcomes are compared with observed ones, and wrong or obsolete entries are corrected. Heuristic estimates H(s) are also learned, as in LRTA*, where H(s) = min over actions of [c(s, a, s') + H(s')], so the rover gradually improves its estimate of the cost to reach the goal.

iv. Online search compared with uninformed and informed search.

Uninformed search (BFS, DFS, UCS): has no goal guidance, expands many irrelevant states, and the algorithms assume they can jump to any generated node, which a physical rover cannot

do. They are inefficient in energy and time. *Informed search (Greedy, A*):* uses a heuristic, so it is far more efficient, but the standard versions assume a known map and expand nodes anywhere in the search space, which is not possible for a robot that has to travel there. *Online search (online DFS, LRTA*, D* Lite):* expands only the node the agent actually occupies, uses what it senses, learns the model and heuristic as it goes, and can backtrack physically; it is therefore designed for unknown and changing environments.

v. Most suitable approach. An informed online search such as LRTA* or D* Lite, supported by a hazard-aware map, is the most suitable. For uncertainty, the belief map is updated with every percept, and unknown cells are assigned a cautious risk cost. For adaptability, when a new rock or crater is sensed the route is repaired incrementally instead of re-planned from scratch. For safety, hazard probabilities and safety margins are added to the cost, reversible actions are preferred, the rover keeps a safe path back, and it stops or waits when the sensing is unreliable. This gives efficient, safe, goal-directed exploration without a complete map.

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. **Clearly define variables, domains, and constraints with explanation**
ii. **Explain how backtracking search works in this scenario**
iii. **Discuss how constraint propagation (e.g., forward checking) improves efficiency**
iv. **Analyze real-world challenges and justify the best strategy for scalability**

Answer:

CSP formulation. The exam timetable is a Constraint Satisfaction Problem defined by variables, domains and constraints.

i. Variables, domains and constraints.

Variables: for every course c, E_c = the exam time slot, and H_c = the exam hall (plus an invigilating faculty member F_c if required).

Domains: D(E_c) = {Day1-Morning, Day1-Afternoon, Day2-Morning, ...} (all available exam slots); D(H_c) = {Hall 1, ..., Hall m} (halls with enough capacity).

Constraints:

1. *No student overlap (binary):* if courses A and B have a common student, then E_A ≠ E_B. This creates a conflict graph between courses.

2. *Limited halls and capacity:* in a given slot, two exams cannot use the same hall, and the number of students in a hall must not exceed its capacity (All-Different on the pair (slot, hall)).

3. *Precedence:* if course A must be held before course B, then E_A < E_B.

4. *Faculty availability (unary and global):* E_c must be in the faculty member's available slots, and one faculty member cannot invigilate two exams at the same time.

5. *Special accommodations:* students needing extra time, a separate room or an accessible hall restrict H_c to suitable halls and may need a longer slot (unary constraints on H_c and the slot length).

Each constraint is explained by the real rule it represents; the unary constraints shrink the domains before the search begins.

ii. Backtracking search. Variables are assigned one at a time. The algorithm picks an unassigned course, tries a value from its domain, and checks that it is consistent with all the assigned courses. If it is consistent it moves to the next course; if no value works, it backtracks to the previous course and tries another value. For example, assigning DBMS to Day1-Morning rules that slot out for every course sharing students with DBMS; if later a course has no valid slot or hall left, the search returns and changes an earlier choice. It is a depth-first search that checks constraints during assignment and is complete, but the worst case is O(d^n). Good ordering heuristics help: MRV (most constrained course first), the degree heuristic, and LCV (least constraining value first).

iii. Constraint propagation (forward checking). After each assignment, forward checking removes the inconsistent values from the domains of the neighbouring (unassigned) variables. If E_DBMS = Day1-Morning, that slot is deleted from the domains of all courses sharing students with DBMS, and the hall is removed from that slot's available halls. If any domain becomes empty, the search backtracks immediately instead of discovering the failure much

later. This early detection of dead ends prunes huge parts of the search tree. Stronger propagation such as arc consistency (AC-3) and, with MRV, forward checking greatly reduces the number of backtracks.

iv. Real-world challenges and best strategy. Challenges: thousands of courses and students give a very dense conflict graph and a huge search space; the problem may be over-constrained (no perfect solution); there are soft preferences (no more than two exams per day, rest gaps, fairness); data errors and last-minute changes (new accommodations, hall unavailable) require rescheduling; and different departments have different needs. Best strategy: use backtracking with MRV + degree + LCV, forward checking/AC-3 and conflict-directed backjumping to satisfy the hard constraints; split the problem by department/cluster (problem decomposition); and use a min-conflicts local search to repair or improve the timetable and to handle changes and soft constraints. This hybrid gives a valid timetable, scales to large universities, and can be updated quickly without rebuilding the whole schedule.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. **Explain how the Minimax algorithm models this problem**

ii. **Analyze the computational challenges of large game trees**

iii. **Explain how Alpha-Beta pruning improves efficiency with detailed reasoning**

iv. **Design an evaluation function and justify the factors considered**

v. **Recommend a strategy for balancing optimality and real-time performance**

Answer:

i. How Minimax models the problem. The game is a two-player, zero-sum, adversarial game. The AI is MAX (it tries to maximize its utility) and the human player is MIN (assumed to play the best counter-moves). The game tree has game states as nodes and moves/actions as edges, and the leaves (terminal or depth-limit states) are given utility values. Values are passed upward:

MINIMAX(s) = max over actions of MINIMAX(result(s, a)) at a MAX node, and min over actions at a MIN node. The AI chooses the move with the highest minimax value, so it gets the best result assuming the opponent also plays optimally. For a real-time strategy game, time is divided into decision steps and units/actions are grouped into abstract moves so that the tree can be modelled.

ii. Computational challenges of large game trees. With branching factor b and depth m, Minimax needs O(b^m) time (space O(bm) with depth-first search). In an RTS many units each have many possible actions, so b is huge. For example, b = 30 and depth 6 gives 30^6 = 729,000,000 positions, impossible within a time limit of a fraction of a second. Further difficulties: non-terminal states need an evaluation function, the horizon effect (a danger just beyond the search depth is missed), real-time (not strictly turn-based) play, and fog of war (hidden information).

iii. How Alpha-Beta pruning improves efficiency. Alpha-Beta keeps two bounds during the search: α = the best value MAX can already guarantee along the current path, and β = the best value MIN can already guarantee. A branch is cut off as soon as $\alpha \geq \beta$, because the remaining children cannot change the final decision. Example: at the root, MAX has found a move with value 6 ($\alpha = 6$). In the next move, MIN finds a reply with value 3, so that move is worth at most 3, which is already worse than 6; the other replies for that move are not examined. Pruned parts could never have influenced the result, so Alpha-Beta returns exactly the same move as Minimax without losing optimality. With the worst move ordering there is no saving, O(b^m); with the best ordering the time is about O(b^(m/2)), i.e. the search can go roughly twice as deep in the same time. Move ordering (captures/strong moves first, killer moves, the best move from the previous iteration, transposition tables) makes pruning effective.

iv. Evaluation function. Since the full tree cannot be searched, the cut-off states are scored by a linear weighted function: Eval(s) = w1·Army strength + w2·Resources/economy + w3·Map/territory control + w4·Technology and upgrades + w5·Base health and safety + w6·Unit positioning and mobility.

Army strength (number, type and health of units) shows the ability to win fights; *resources and income* show the long-term ability to build; territory control gives vision and access to resources; technology shows future power; base health detects the risk of losing the game; positioning captures tactical advantage such as chokepoints. The function must be cheap to compute, the weights are tuned by testing or self-play (machine learning), and values are

normalized; for unstable positions (fighting in progress) the search is extended (quiescence search).

v. Strategy balancing optimality and real-time performance. Use depth-limited Alpha-Beta search with iterative deepening as an anytime algorithm: search to depth 1, 2, 3... until the time budget ends, and play the best move from the last completed depth, so a valid move is always ready. Add good move ordering (using the previous iteration's best move), transposition tables, action abstraction (grouping units into macro-actions), selective/forward pruning (beam search) for unpromising moves, and quiescence search to reduce the horizon effect. Allocate more time to critical situations and less to simple ones, and use a fallback heuristic or scripted move if time runs out. This gives strong moves that are near-optimal and always within the real-time limit.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand